

Pyroclast: GPU-Accelerated Monte Carlo Simulation for Lava-Flow Habitat Impact Assessment

Emanuele Galiano

May 24, 2026

Contents

1	Kernel Monte Carlo	2
1.1	Mathematical Formulation	2
1.1.1	Variables and Parameters	2
1.1.2	The Monte Carlo Run	2
1.1.3	Monte Carlo Estimator	3
1.2	Standard Kernel	3
1.2.1	Kernel Design	3
1.2.2	Algorithmic Complexity	4
1.3	1D Ping-pong Kernel	5
1.3.1	Double buffering	5
1.3.2	Trade-offs	5
1.3.3	Algorithmic Complexity	6
1.4	Transitioning to 2D Architectures	7
1.5	TODO: Benchmark	7
A	Library Architecture	8

Chapter 1

Kernel Monte Carlo

In this chapter we present the design and implementation of the kernel Monte Carlo method used in Pyroclast for simulating lava flow and assessing habitat impact. In the first section we discuss the mathematical formulation...

1.1 Mathematical Formulation

The problem consists of estimating the probability that a lava flow will cause the destruction of a specific habitat area, given a pre-calculated map of invasion probabilities.

1.1.1 Variables and Parameters

We can define the variables of the problem as follows:

- Let N_c be the total number of active cells that make up the habitat under evaluation.
- Let $\mathcal{Q} = \{p_0, p_1, \dots, p_{N_c-1}\}$ be the set of invasion probabilities for each cell of the habitat, where p_i is the probability that the lava flow will invade cell i .
- Let $\theta \in [0, 1]$ be the critical fraction or destruction threshold, which represents the minimum fraction of the habitat that must be invaded for it to be considered destroyed.
- Let R be the total number of Monte Carlo simulations to be performed.

1.1.2 The Monte Carlo Run

For a single simulation identified by the index r , where $0 \leq r < R$, we perform the following steps:

1. **Sampling:** For each individual cell k of the habitat, we draw a pseudo-random number $X_{r,k} \sim \mathcal{U}(0, 1)$.
2. **Invasion Evaluation:** Cell k is considered "invaded" by lava in simulation r if the draw value is less than or equal to its invasion probability, i.e., if $X_{r,k} \leq p_k$, and it can be formalized with an indicator function

$$I_{r,k} = \begin{cases} 1 & \text{if } X_{r,k} \leq p_k \\ 0 & \text{otherwise} \end{cases} \quad (1.1)$$

3. **Destruction Assessment:** We first define the total number of invaded cells in simulation r as

$$C_r = \sum_{k=0}^{(N_c-1)} I_{r,k} \quad (1.2)$$

and then the habitat is considered destroyed in run r if the fraction of invaded cells exceeds the critical threshold θ (normalized by the total number of cells):

$$D_r = \begin{cases} 1 & \text{if } \frac{C_r}{N_c} \geq \theta \\ 0 & \text{otherwise} \end{cases} \quad (1.3)$$

1.1.3 Monte Carlo Estimator

The ultimate goal of the parallel execution is to compute the total number of simulations that resulted in habitat destruction. This requires summing D_R , over all R runs. The Monte Carlo estimator for the probability of habitat destruction, also the overall probability of destruction $\hat{P}_{\text{destruction}}$, can be estimated as:

$$\hat{P}_{\text{destruction}} = \frac{1}{R} \sum_{r=0}^{R-1} D_r \quad (1.4)$$

1.2 Standard Kernel

The first version of the Monte Carlo kernel implemented in Pyroclast is a straightforward parallelization of the algorithm described in the previous section. Each simulation run is executed independently, and the results are aggregated by a reduction operation at the end of the kernel execution.

1.2.1 Kernel Design

The kernel operates on a 1-dimensional grid of work-items, where each work-item is responsible for executing a whole Monte Carlo simulation on the full array of invasion probabilities.

Work-Item Mapping. A critical architectural decision in this baseline was how to map the computational workload to the GPU's work-items. The most intuitive approach is the pure stream processing model, where each work-item is responsible for executing only single Monte Carlo. This approach was deliberately discarded due to a severe hardware inefficiency known as the "prime number problem"¹: in OpenCL the number of launched work-items is defined by the Global Work Size, which is hardware-partitioned into smaller blocks called work-groups, defined by the Local Work Size. If we tie the number of work-items to the number of simulations, an user requesting a number of simulations that is prime would force a primw GWS and since the only integer divisors of a prime number are 1 and itself, the hardware would be forced to launch work-groups containing a single work-item (LWS = 1).

¹Also knowns as one of the "leaky abstractions" of GPU programming.

Total Monte Carlo Simulations (R)

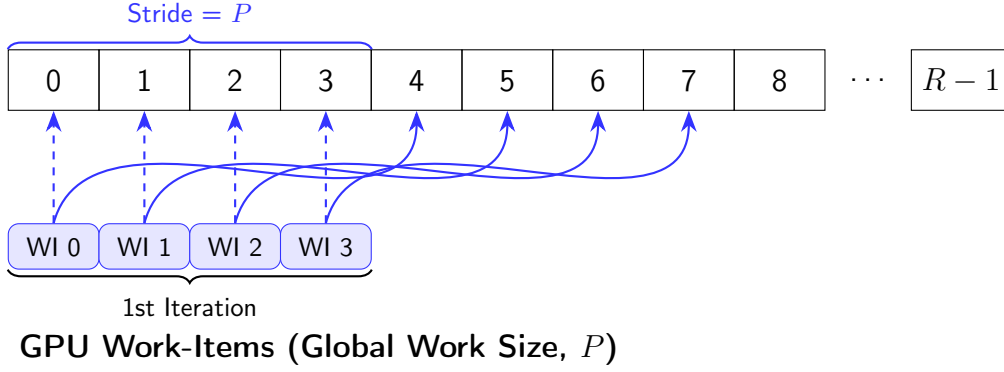


Figure 1.1: Visual representation of the Grid-Stride Loop (Sliding Window) approach. A hardware-optimized grid of P work-items iteratively processes the total R simulations by advancing with a stride equal to the Global Work Size.

Sliding Window. To resolve this limitation a sliding window approach is adopted. Instead of launching a grid sized perfectly to the dataset, we launch a hardware optimized grid and allow each work-item to process multiple runs through a for loop, taking strides² equal to the entire GWS.

1.2.2 Algorithmic Complexity

The theoretical time complexity of the kernel depends on the total number of Monte Carlo simulations R , the number of cells N_c , the number of work-items P launched and the dimension of the work-groups L (LWS).

Time Complexity. The time complexity of the kernel can be divided in due principal components:

Sampling and Monte Carlo Run: Kernel doesn't map a single work-item, so the for loop iterates over runs for a total of R/P iterations. For each run, the kernel iterates over the N_c cells of the habitat in order to evaluate the invasion, resulting in a time complexity

$$\mathcal{O}\left(\frac{R}{P} \cdot N_c\right)$$

Reduction: After all work-items have completed their assigned runs, a reduction operation is performed to sum the results of all simulations. The tree reduction operates on an array in local memory of size L (the number of work-items in a work-group) and since LWS is a power of 2, the reduction divides the problem size by 2 at each step, resulting in a time complexity of

$$\mathcal{O}(\log_2 L)$$

²The stride represents the number of simulations processed by each work-item in the sliding window approach.

Finally, the overall time complexity of the kernel can be expressed as:

$$T(N_c, R, P, L) = \mathcal{O}\left(\frac{R}{P} \cdot N_c + \log_2 L\right) \quad (1.5)$$

Space Complexity. The space complexity of the kernel is determined in hierarchies:

- **Global Memory:** The kernel reads the compacted array of probabilities `p_vec`, which has a size of $\mathcal{O}(N_c)$. In output, it writes a partial sum of the destruction results not for each work-item, but for each work-group. Assuming a global work size of P and a local work size of L , the output array `partial` has a size of P/L . Thus, the total global memory complexity is $\mathcal{O}(N_c + P/L)$.
- **Local Memory:** Each work-group allocates a shared `scratch` array used for the parallel tree reduction. The size of this array corresponds exactly to the local work size L (which is statically set to 256). Therefore, the local memory complexity is $\mathcal{O}(L)$ per work-group.
- **Private Memory:** Each individual work-item maintains only a few local state variables (such as the `private_sum` accumulator, loop counters, and thread IDs). This requires a strictly constant amount of memory, resulting in a private space complexity of $\mathcal{O}(1)$ per work-item.

1.3 1D Ping-pong Kernel

While the standard 1D kernel performs an efficient parallel tree reduction, it updates the shared memory array in-place. This causes the use of two barriers:

- One before writing in local memory, to ensure that all work-items have completed their computations and are ready to write their partial sums.
- One after writing, to ensure that all work-items have completed their writes before any work-item can read from the shared memory for the reduction step.

1.3.1 Double buffering

In order to eliminate the need for the second barrier, a double buffering technique is adopted. Instead of using a single shared memory array for the reduction, two separate arrays are allocated: one for reading and one for writing. During each step of the reduction, work-items read from one buffer and write their results to the other buffer. After each step, the roles of the buffers are swapped. This approach allows work-items to proceed with their computations without waiting for all other work-items to complete their writes, thus reducing synchronization overhead.

1.3.2 Trade-offs

The implementation of this kernel explicitly highlights the trade-off between memory usage and synchronization overhead.

Benefits. By guaranteeing that the buffer being read from the source is physically distinct from the buffer being written to the destination, we can eliminate any possibility of Write-After-Read hazards. This allows the step to be safely executed with only a single barrier at the top of the loop to publish the initial writes, potentially allowing work-items to proceed with their computations without waiting for all other work-items to complete their writes, thus reducing synchronization overhead.

The cost. To achieve this safety, the kernel requires an additional local array. This part we'll be seen in the space complexity analysis at 1.3.3, but it is important to note that this additional memory usage may limit the maximum local work size that can be used, as the total local memory available on the GPU is a fixed resource. This could potentially reduce the degree of parallelism and impact performance if the local work size needs to be reduced to accommodate the additional buffer.

1.3.3 Algorithmic Complexity

The theoretical time complexity of the 1D Ping-Pong kernel remains fundamentally similar to the standard version, as it depends on the total number of Monte Carlo simulations R , the number of cells N_c , the number of launched work-items P , and the dimension of the work-groups L (LWS).

Time Complexity. The time complexity of the kernel is divided into the same two principal components as the standard kernel and the time complexity of each component remains unchanged:

$$T(N_c, R, P, L) = \mathcal{O}\left(\frac{R}{P} \cdot N_c + \log_2 L\right) \quad (1.6)$$

Space Complexity. The space complexity highlights the main architectural trade-off of the double-buffering approach:

- **Global Memory:** The kernel reads the compacted `p_vec` array of size $\mathcal{O}(N_c)$ and writes to the `partial` array of size P/L . The total global memory complexity remains $\mathcal{O}(N_c + P/L)$.
- **Local Memory:** To eliminate synchronization hazards with fewer barriers, each work-group allocates *two* shared arrays instead of one: a primary `scratch1` array of size L and a secondary `scratch2` array of size $L/2$. While asymptotically this is still $\mathcal{O}(L)$ per work-group, the absolute footprint is strictly $1.5 \times L$. This increased local memory usage directly reduces the maximum hardware occupancy (number of active work-groups) on the Compute Units.
- **Private Memory:** Each work-item maintains the usual local state variables alongside a new `val` register used to carry the running sum across loop iterations safely. This still requires a constant amount of memory, resulting in a private space complexity of $\mathcal{O}(1)$ per work-item.

1.4 Transitioning to 2D Architectures

1.5 TODO: Benchmark

DUMMY CIT [1]

Note e appunti.

Cose da vedere:

- Il numero di work-items e work-groups lanciati per ogni kernel, e come questo si relaziona con il numero totale di simulazioni R e la dimensione del lavoro locale L . Soprattutto per fare vedere la differenza tra due barriere e ping pong.

Appendix A

Library Architecture

(Appendix placeholder — to be expanded.)

Bibliography

- [1] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, apr 2009.